

CS 499 Milestone Two: Software Design and Engineering

Artifact Enhancement Narrative

Samuel Raymond Kwibe

CS 499 Computer Science Capstone

Southern New Hampshire University

Section One: Describing the Artifact

When thinking about which project to use for this capstone, the MunchiesSNHU Food Waste Tracker felt like the most honest choice. It was not the cleanest project or the most finished one, but it was the one that felt most like real software — messy in some places, ambitious in others, and full of ideas that never quite got connected together. The project was originally built during an earlier course in the Computer Science program at Southern New Hampshire University. The goal behind it was to create a web application that a college campus could use to track food waste, monitor compost bins, log environmental sensor readings like temperature, humidity, pH, and gas levels, and eventually generate sustainability reports that staff and administrators could act on.

The application has two main parts. The frontend is a collection of HTML pages with JavaScript and CSS that handles everything the user sees — login pages, signup, dashboards, data entry forms, and report screens. The backend is built with Node.js and Express and connects to a MongoDB database. It handles user accounts, authentication using JWT tokens, food waste records, and API routes that the frontend calls to get and send data. There are multiple user roles in the system — regular users, staff, and admins — and different parts of the application are supposed to be restricted based on those roles. On paper, the project had everything a real application needs. The problem was that many of those pieces were not actually talking to each other yet.

Section Two: Why This Artifact Was Chosen

Choosing this artifact was not an easy decision at first. Looking at it honestly, the project had a lot of rough spots. Some routes were stubs that did nothing. Some controllers had working code

but no routes to reach them. The frontend and backend had field name mismatches that would have caused silent failures. There were empty files sitting in the project that served no purpose. For a moment, it felt like choosing this project was setting things up to fail.

But that is actually exactly why it became the right choice. A capstone portfolio is not supposed to show a finished, polished product that required no effort. It is supposed to show the ability to look at something critically, identify what is wrong, and fix it in a way that demonstrates real understanding. The MunchiesSNHU project gave plenty of material to work with. It had enough structure to build on and enough problems to solve in a meaningful way.

The specific components that make this artifact valuable for demonstrating software design and engineering skills are the route architecture, the controller structure, the authentication system, the security middleware, and the overall organization of the backend. These are not surface-level features — they represent the decisions that determine whether a web application actually works correctly and safely when real users interact with it. Fixing and improving those components in this milestone shows a genuine understanding of how professional backend systems are built.

Section Three: What Was Actually Improved and How

Before diving into the work, a Git branch called milestone-two was created from the master branch. This was done so the original code would stay completely untouched. The professor needs to see a before and after comparison, and the before-enhancement folder in the repository serves as that original snapshot. All changes in this milestone were made only on the milestone-two branch, so there is a clean, verifiable record of what existed before and what was changed. Here is a breakdown of every improvement that was made during this phase:

- The first thing tackled was the three controllers that had working code but no route files connecting them to the application. The `staffController`, `UserController`, and `notificationController` all had real logic written inside them — functions for getting staff lists, managing users, and handling notification preferences — but none of those functions could ever be reached because no route file existed to map them to an API endpoint. Three new route files were created: `staffRoutes.js`, `userRoutes.js`, and

notificationRoutes.js. Each route was protected with the JWT middleware so only authenticated users with the right role could access them. Then those route files were mounted in app.js using app.use() so the server would actually recognize and handle those requests. This sounds straightforward, but getting the authorization logic right for each route — deciding which endpoints needed admin access versus just a logged-in user — required reading through the controllers carefully and understanding what each function was supposed to do.

- The second improvement was refactoring the food waste route file to follow proper MVC structure. Before this change, the POST, PUT, and DELETE handlers for waste entries were written directly inside foodWasteRoutes.js. That works, but it mixes routing logic with business logic in the same file, which makes the code harder to read, harder to test, and harder to maintain. Those handlers were moved into wasteEntryController.js as proper controller functions, and the route file was updated to just call those functions. The route file became thin and the controller became the place where the actual work happens. This is a standard software engineering practice, and making that change showed a real understanding of why MVC separation matters — not just that it is supposed to be done, but what problem it actually solves.
- The third improvement was fixing the generateToken.js utility file. Before this milestone, the function for generating JWT tokens was written directly inside authController.js, and there was also a generateToken.js file in the utils folder that existed as an empty export — literally a file with no code in it. That empty file was confusing because it looked like something was supposed to be there but never got written. The generateToken function was moved from authController.js into generateToken.js as a proper reusable utility, and authController.js was updated to import it from there. This removed the duplication and made the codebase cleaner and easier to follow.
- The fourth area of work was the placeholder controller functions that were returning 501 Not Implemented errors. Several functions across different controllers — createEntry,

updateEntry, deleteEntry, createTip, recordReading, and a few others — had been written as stubs that just sent back a 501 response with no real logic. Some of those were replaced with real implementations that actually save, update, or delete records from the database. Others, particularly ones for features that are genuinely planned for later phases, were kept as 501 responses but updated so the response format is consistent and professional across the entire codebase. Previously some stubs sent back plain text, some sent JSON, and some sent nothing meaningful at all. Having consistent error response formatting across all routes is a software design principle that makes the API much easier to work with.

- The fifth improvement was deleting the empty SensorData.js model file. This file existed in the models folder but had absolutely zero content — not even a basic schema structure. An empty model file is worse than not having the file at all because it creates confusion about whether sensor data is being stored somewhere or not. Removing it cleaned up the project and made the actual models in the folder easier to find and understand.
- The sixth and final improvement for this milestone was going through all eleven modified and created files and adding student-style code comments throughout. Every section of code now has comments that explain what the code does, why it is structured the way it is, and what would happen if that piece were removed. The goal was not to comment every single line robotically but to explain the thinking behind each block so that someone reading the code for the first time — including the professor reviewing this submission — can follow the logic without needing prior knowledge of the project.

Section Four: Course Outcomes and Whether They Were Met

Going back to the plan that was laid out in Milestone One during the code review, the goal was to demonstrate progress toward several CS 499 course outcomes through the software design and engineering enhancements. Looking at what was actually completed, here is an honest assessment of how that went.

Course Outcome Four — the one about using well-founded techniques and tools to implement computing solutions — was the main target for this milestone, and it was fully addressed. The MVC refactoring, the new route files, the utility module extraction, and the placeholder cleanup are all direct examples of applying standard software engineering practices to a real codebase. These are not theoretical concepts — they are practical changes that make the application more maintainable, more readable, and more professional.

Course Outcome Five — developing a security mindset — was also addressed, though much of that work had actually already been done in the codebase before this milestone began. The security middleware including Helmet for HTTP headers, rate limiting to prevent API abuse, MongoDB sanitization to block injection attacks, and XSS protection were all already wired into `app.js`. The JWT protect and authorize middleware was already active on the food waste, feedback, and tip routes. What this milestone contributed to security was extending that same protection to the three newly mounted route files, so no new endpoints were left unprotected.

Course Outcome Two — designing and delivering professional-quality communications — was addressed through the code comments added to all eleven files. Writing comments that explain technical decisions clearly and consistently to a technical audience is a form of professional communication that matters just as much as any written document.

The remaining outcomes around algorithmic design and database optimization were not the focus of this milestone and will be addressed in the next two phases of the course as planned.

Section Five: Honest Reflection on the Process and What Was Learned

This milestone did not go exactly the way it was expected to go, and that turned out to be one of the most valuable parts of the whole experience. When the code review was done in Milestone One, six specific problems were identified that needed to be fixed. Going into the enhancement phase, the assumption was that all six of those problems would need to be built from scratch. The plan was laid out, the message was sent to the developer, and the expectation was that there would be a lot of new code to write.

Then the developer came back and said that five of the six issues were already resolved in the current codebase. The routes were already fixed. The field names were already aligned. Firebase had already been removed from signup. The security middleware was already active. The JWT protection was already on the routes. At first, that felt almost like a problem — if most of the planned work was already done, would the enhancement still count for the assignment?

But thinking through it more carefully, that situation actually taught something more useful than any of the planned fixes would have. It showed the difference between reviewing code statically — reading files and making assumptions — versus understanding how those files actually connect and run together. The code review identified problems based on reading individual files in isolation. But some of those problems had already been fixed in other parts of the codebase that were not visible during the initial review. That is a real-world scenario that happens constantly in professional software development. Code reviews catch things, but they do not always see the full picture.

The remaining work — the three unmounted controllers, the placeholder stubs, the token utility, and the empty file — was genuinely new work that required careful thought. The most difficult part was the MVC refactoring of the food waste routes. Moving logic from a route file into a controller seems simple, but it required understanding exactly what each function was doing, making sure the request and response objects were being handled correctly in the new location, and confirming that the route file still called the controller methods with the right parameters. One mistake in that process would have silently broken the food waste entry feature, which is one of the core functions of the entire application.

Another challenge that came up unexpectedly was deciding what to do with the placeholder 501 stubs that represented genuinely unfinished features. The temptation was to just write something — anything — to get rid of the 501 response. But writing incomplete or incorrect logic just to fill a gap would have been worse than leaving the stub in place. The better decision was to make those stubs consistent and clear so that anyone reading the code would immediately understand that the feature is planned but not yet implemented, rather than wondering if something was

accidentally broken. That kind of decision-making — knowing when not to write code — is something that only comes from thinking about the bigger picture of how the application is supposed to work.

Overall, this milestone felt like the first time the project started to resemble something that could actually be deployed and used. The before-enhancement snapshot shows a project that had the right ideas but was only halfway connected. The milestone-two branch shows a project where the structure is clean, the routes are complete, the controllers are doing their jobs, and the security layer is consistent across the entire backend. That transformation is what software design and engineering is really about.

AI Usage Disclosure

An AI writing assistant was used to help organize and draft portions of this narrative. All content was reviewed, edited, and rewritten to reflect personal understanding, direct experience working through the artifact enhancements, and honest reflection on the challenges and lessons learned throughout this milestone.

References

Fowler, M. (2018). Refactoring: Improving the design of existing code (2nd ed.). Addison-Wesley.

Martin, R. C. (2008). Clean code: A handbook of agile software craftsmanship. Prentice Hall.

Mozilla Developer Network. (2023). Express web framework (Node.js/JavaScript).

https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs